

Overview of .NET Framework

What is .NET Framework?

The **.NET Framework** is a **software development platform** developed by **Microsoft**, primarily for building and running applications on Windows.

It provides a **controlled programming environment** where software can be developed, installed, and executed on Windows-based operating systems.

Key Components:

1. Common Language Runtime (CLR):

- The runtime environment in the .NET Framework.
- Handles memory management, security, exception handling, and more.
- Acts like a virtual machine (similar to JVM in Java).

2. .NET Class Library:

- A vast collection of pre-built classes, interfaces, and value types.
- Provides support for file I/O, database access, XML parsing, and more.

3. ASP.NET:

- A part of the .NET Framework used to build dynamic web applications and services.

4. Windows Forms and WPF:

- For building rich desktop applications.

Origin of .NET Technology

In the late 1990s, Microsoft faced increasing pressure from the rise of the internet and Java, which promised “**write once, run anywhere.**” Microsoft wanted a platform that could:

- Provide a **unified environment** for application development.
- Compete with Java.
- Integrate well with Windows.

Birth of .NET:

- Microsoft introduced **.NET Framework** in **2002**, along with **C#**.
- Designed to support **multiple languages** (C#, VB.NET, F#, etc.) through the **Common Language Runtime (CLR)**.
- Initially targeted **Windows** applications (desktop, web).

Evolution:

- .NET Framework was succeeded by **.NET Core** (2016) — a cross-platform, open-source version.
- Today, we have **.NET 5+ (Unified .NET)** that merges .NET Framework and .NET Core into a single platform supporting Windows, macOS, Linux, mobile, cloud, and more.

The .NET Framework:

The **.NET Framework** is a Microsoft platform that provides a runtime environment (CLR) and a rich set of libraries (FCL) to develop and run Windows applications in a secure, reliable, and efficient way.

Main Components

1. CLR (Common Language Runtime):

- The heart of .NET Framework.
- Manages program execution.
- Provides services like memory management, garbage collection, exception handling, and security.

2. FCL (Framework Class Library):

- A vast collection of pre-written classes, methods, and APIs.
- Helps developers perform tasks like database access, file operations, graphics, and networking.

Key Features

- **Language Interoperability** → Supports multiple languages (C#, VB.NET, F#).
- **Managed Code Execution** → Programs run inside the CLR for safety and reliability.
- **Automatic Memory Management** → Garbage collector frees unused memory.
- **Rich Class Library** → Reduces coding effort by reusing ready-made classes.
- **Security** → Provides role-based and code-access security.

Uses

- Building **Windows desktop applications**
- Creating **web applications** (ASP.NET)
- Developing **enterprise-level applications**
- Building **Windows services**

Major Components of the .NET Framework

1. Common Language Runtime (CLR)

The **heart of the .NET Framework**, it provides the execution environment for all .NET applications.

Key Responsibilities:

- **Memory management** (garbage collection)

- **Exception handling**
- **Security** (code access security)
- **Thread management**
- **Type safety**
- **Just-In-Time (JIT) compilation** – compiles MSIL to native machine code

2. .NET Framework Class Library (FCL/BCL)

A large collection of **reusable classes, interfaces, and types** used to build applications.

Key Namespaces:

- System – core functionality (base types, arrays, strings)
- System.IO – file and stream input/output
- System.Net – network programming
- System.Data – data access (ADO.NET)
- System.Xml – XML processing
- System.Windows.Forms or System.Web – for UI (Windows/web)

3. Common Type System (CTS)

Defines how **types are declared and used** in the .NET Framework.

Purpose:

- Ensures all .NET languages share the same data types
- Allows cross-language integration and interoperability

Example: An int in C# is the same as an Integer in VB.NET under CTS.

4. Common Language Specification (CLS)

A set of **rules and guidelines** that languages must follow to be compatible with the .NET Framework.

Purpose:

- Promotes **language interoperability**
- Allows code written in one .NET language (e.g., C#) to be used in another (e.g., VB.NET)

5. Assemblies and the Global Assembly Cache (GAC)

- **Assembly**: A compiled unit of code (.dll or .exe) containing metadata, MSIL, and resources.

- **GAC:** A machine-wide code cache that stores **shared assemblies** for reuse across applications.

6. Windows Forms / WPF / ASP.NET / ADO.NET

These are application-specific libraries built on top of the core class library.

Technology	Use Case
Windows Forms	GUI-based desktop apps
WPF (Windows Presentation Foundation)	Rich media desktop apps
ASP.NET	Dynamic web applications
ADO.NET	Data access and database interaction

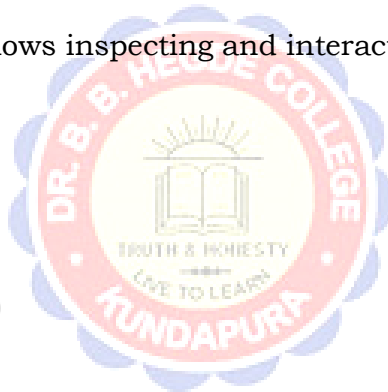
7. Metadata and Reflection

- **Metadata:** Information about the types, members, and references in an assembly.
- **Reflection:** A feature that allows inspecting and interacting with metadata at runtime.

8. Security Framework

Includes:

- **Code Access Security (CAS)**
- **Role-based security**
- Helps ensure applications are **safe and restricted** from performing unauthorized actions.



9. Language Interoperability

Thanks to **CLR, CTS, and CLS**, multiple languages (C#, VB.NET, F#, etc.) can be used together in the same application.

10. Managed Code vs Unmanaged Code

- **Managed Code:** Code that runs under CLR supervision.
- **Unmanaged Code:** Native code (e.g., C/C++) that does not run under CLR.

1. Common Language Runtime (CLR)

What is CLR?

The **Common Language Runtime (CLR)** is the **core execution engine** of the .NET Framework. It runs the code and provides services that make development easier and safer.

Main Responsibilities of CLR:

1. **Memory Management** – Uses **garbage collection** to automatically manage memory.
2. **Code Execution** – Runs the compiled **Intermediate Language (IL)** code by converting it to machine code via **Just-In-Time (JIT)** compilation.
3. **Exception Handling** – Provides a consistent model to handle errors.
4. **Security** – Controls access and permissions through **Code Access Security (CAS)**.
5. **Thread and Process Management** – Manages application threads and handles multitasking.
6. **Type Safety** – Ensures code accesses only authorized memory locations.

In simple terms: The CLR makes sure your .NET program **runs safely, efficiently, and correctly**.

2. Common Type System (CTS)

What is CTS?

The **Common Type System (CTS)** defines a **set of data types** and rules that all .NET languages must follow.

This ensures that code written in one language can work with code written in another.

Why CTS is Important:

- **Supports language interoperability** (e.g., using C# code in VB.NET and vice versa).
- Ensures **type compatibility** across different languages.
- Helps prevent **type-related runtime errors**.

Example:

- C# has int, VB.NET has Integer – but both are treated as System.Int32 under CTS.
- All .NET types derive from the base type **System.Object**.

3. Common Language Specification (CLS)

What is CLS?

The **Common Language Specification (CLS)** is a **set of rules** or guidelines defined by Microsoft. These rules specify which features and data types **must be supported** by all .NET languages to ensure **cross-language compatibility**.

Purpose of CLS:

- Allows libraries and code to be **shared across different .NET languages**.

- Helps developers write **language-neutral code**.

Example:

- C# supports unsigned int (uint) — but not all languages (like VB.NET) support it.
- So, using uint is **not CLS-compliant** and can cause problems in cross-language development.

Managed Code**What is Managed Code?**

Managed code is the code that is **executed under the control of the Common Language Runtime (CLR)** in the .NET Framework.

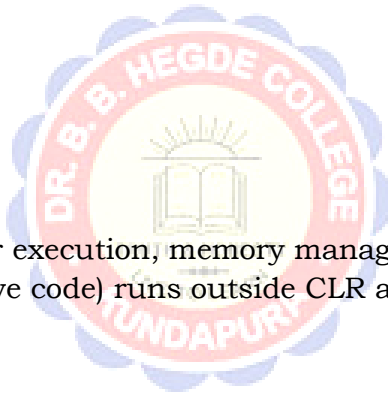
When you write code in a .NET-supported language like C#, VB.NET, or F#, the compiler compiles it into **Intermediate Language (IL)**. This IL code is then **managed and executed** by the CLR.

Key Features of Managed Code:

- **Automatic memory management** (via Garbage Collection)
- **Type safety**
- **Security enforcement**
- **Exception handling**
- **Cross-language support**

Managed code relies on the CLR for execution, memory management, and security.

Unmanaged code (e.g., C/C++ native code) runs outside CLR and must manage its own memory.

**2. Assemblies****What is an Assembly?**

An **assembly** is the **basic building block of a .NET application**.

It contains **compiled code (IL)**, **metadata**, and **resources** (images, strings, etc.) needed to run the program.

Types of Assemblies:

- **Private Assembly:** Used only by a single application.
- **Shared Assembly:** Can be used by multiple applications; usually stored in the **Global Assembly Cache (GAC)**.

Contents of an Assembly:

- **Manifest:** Information about the assembly (name, version, culture, etc.)
- **IL Code:** The actual code to be executed (in Intermediate Language).
- **Metadata:** Describes types, methods, and references used in the code.
- **Resources:** Images, strings, XML files, etc.

Assemblies are usually .dll (library) or .exe (application) files.

3. Intermediate Language (IL)

What is IL?

Intermediate Language (IL), also known as **MSIL** (Microsoft Intermediate Language), is the **platform-independent code** that .NET languages are compiled into.

Your C#/VB.NET code → Compiled into IL → Then executed by CLR.

IL Characteristics:

- It's **not machine-specific**
- It's **closer to human-readable** than native machine code
- It gets compiled to native code by the **JIT compiler**

IL allows your application to be **platform-independent** up to the point of execution.

4. Just-In-Time (JIT) Compilation

What is JIT Compilation?

Just-In-Time (JIT) Compilation is the process where the **CLR compiles IL code into machine code** just before it is executed.

How JIT Works:

1. You write code in C#.
2. It's compiled to IL and stored in an assembly.
3. When the program runs, the CLR uses **JIT compiler** to convert IL into **native machine code**.
4. That native code is then executed on your computer.

Types of JIT Compilers:

- **Normal JIT**: Compiles code as needed, method by method.
- **Pre-JIT (Ngen.exe)**: Compiles the whole application into native code in advance.
- **Econo-JIT** (used in older versions): Optimizes memory usage for small devices.

JIT helps optimize performance by only compiling code when needed.

.NET Framework Base Class Library (BCL)

What is the Base Class Library?

The **Base Class Library (BCL)** is a **large collection of reusable classes and types** built into the .NET Framework.

It provides **fundamental building blocks** for creating .NET applications.

BCL is a part of the **larger .NET Framework Class Library (FCL)**, but focuses on the **core and most commonly used functionalities**.

Key Features of BCL:

- Pre-written and tested code by Microsoft.
- Saves time—developers don't need to write common functions from scratch.
- Provides support for core programming tasks like:
 - Data types (int, string, DateTime, etc.)
 - Collections (List, Dictionary)
 - File I/O
 - Networking
 - Security
 - Threading
 - Reflection
 - Exception handling

Common Namespaces in BCL:

Namespace	Purpose
System	Core classes (data types, exceptions, console)
System.IO	File input/output
System.Net	Networking and internet protocols
System.Collections	Data structures like arrays, lists, stacks
System.Threading	Multithreading and parallel programming
System.Reflection	Metadata and dynamic type information

The BCL makes application development in .NET **easier, faster, and more reliable**.

2. Visual Studio .NET

What is Visual Studio .NET?

Visual Studio .NET (now just called **Visual Studio**) is Microsoft's **Integrated Development Environment (IDE)** for building applications using the .NET Framework.

It was first introduced with the .NET Framework in the early 2000s.

Key Features of Visual Studio:

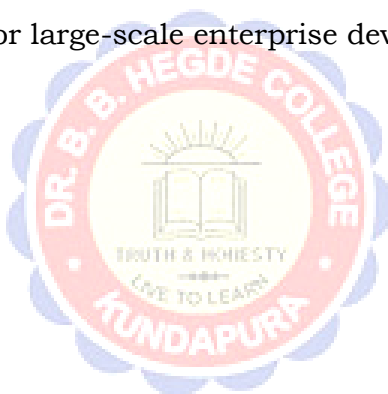
Feature	Description
Code Editor	Rich editor with syntax highlighting, IntelliSense, and refactoring tools
Designer Tools	Drag-and-drop UI designers for Windows Forms, WPF, and ASP.NET
Compiler and Debugger	Compile and debug .NET applications directly inside the IDE
Project Templates	Built-in templates for C#, VB.NET, ASP.NET, WPF, etc.
NuGet Package Manager	Manages external libraries and packages
Version Control	Integration with Git, GitHub, and Azure DevOps
Multi-language Support	Write and manage projects in C#, VB.NET, F#, etc.

Editions of Visual Studio:

- **Community** (free for individuals and small teams)
- **Professional** (paid, with more features)
- **Enterprise** (advanced tools for large-scale enterprise development)

It is designed to be:

- **Simple**
- **Powerful**
- **Type-safe**
- **Flexible**
- **Component-oriented**



C# is similar in syntax to C++, Java, and JavaScript, but it is deeply integrated with the .NET Framework (and now .NET Core / .NET 5+).

CHAPTER 2

OVERVIEW OF C#

Introduction to C# (C-Sharp)

What is C#?

C# is a **modern, object-oriented, and type-safe programming language** developed by **Microsoft** in the early 2000s as part of its .NET initiative.

It is:

- Similar in syntax to Java and C++
- Designed to be simple and efficient
- The primary language used for building .NET applications

Features of C#:

- Object-Oriented Programming (OOP)
- Automatic garbage collection
- Strong type checking
- Properties and indexers
- LINQ (Language Integrated Query)
- Asynchronous programming with `async/await`

Introduction

C# (pronounced as C sharp) is a new programming language developed by Microsoft as a part of their .NET strategy to provide web-based services. It is a fully object-oriented language like Java and is the first component oriented language. The major highlights of the language are

- ✓ It is derived from C/C++ family
- ✓ It is the only component-oriented language
- ✓ It is designed for .NET framework
- ✓ It combines the best features of commonly used languages : productivity of Visual Basic, the power of C++ and elegance of Java
- ✓ Major parts of .NET are coded in C#

Example: C# Console Application

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;
```

```
namespace Annappa  
{  
    class Program  
    {
```

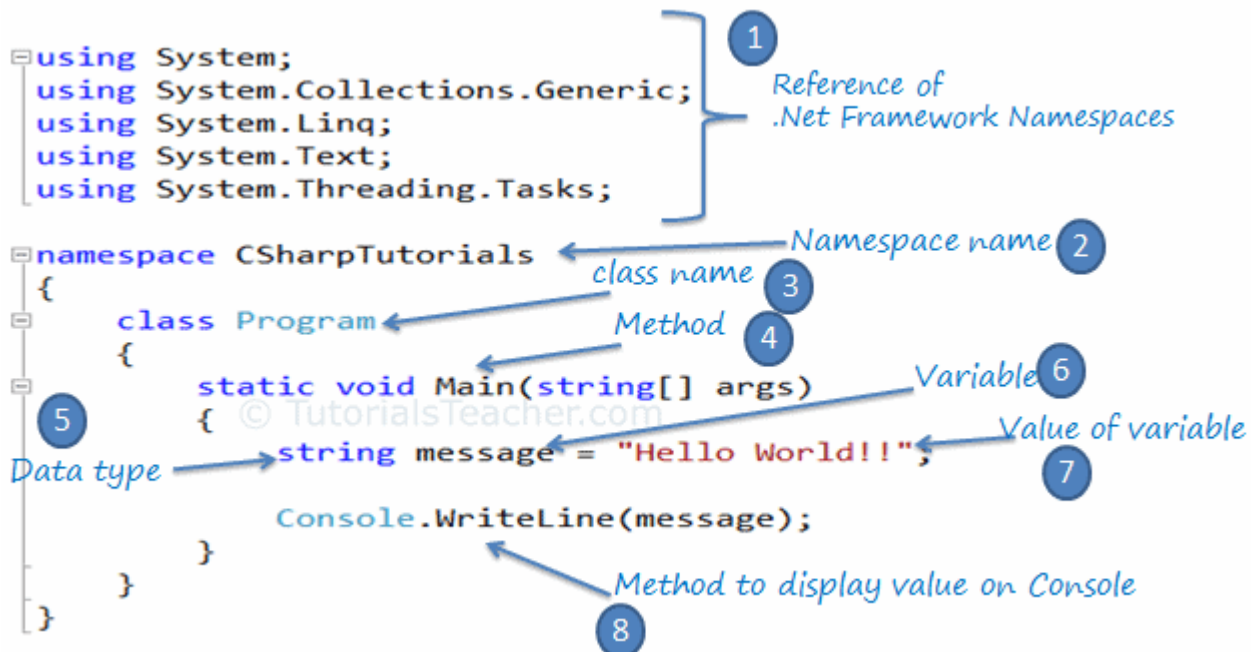
```

static void Main(string[] args)
{
    string message = "Hi Friends, This is my First program in C# ";

    Console.WriteLine(message);
    Console.Read();
}
}
}

```

Let's understand the above C# structure.



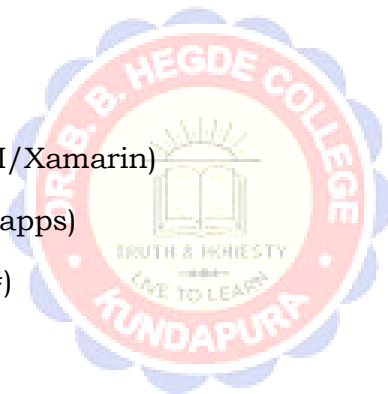
1. Every .NET application takes the reference of the necessary .NET framework namespaces that it is planning to use with the using keyword, e.g., using System.Text.
2. Declare the namespace for the current class using the namespace keyword, e.g., namespace Namespace Sona
3. We then declared a class using the class keyword: class Program
4. The Main() is a method of Program class is the entry point of the console application.
5. String is a data type.
6. A message is a [variable](#) that holds the value of a specified [data type](#).
7. "Hello World!!" is the value of the message variable.
8. The Console.WriteLine() is a static method, which is used to display a text on the console.

Key Characteristics of C#

Feature	Description
Object-Oriented	Supports classes, inheritance, polymorphism, encapsulation, abstraction
Component-Based	Supports modular and reusable development using assemblies
Strongly Typed	Variables must be declared with data types; prevents many common bugs
Modern Syntax	Clean and easy-to-read code structure
Interoperable	Can work with other .NET languages (VB.NET, F#) via the CLR
Type Safe	Prevents operations that could lead to memory corruption
Versatile	Used for desktop apps, web apps, APIs, games, mobile apps, and cloud apps

What Can You Build with C#?

- Windows Applications (WinForms, WPF)
- Web Applications (ASP.NET)
- Web APIs (RESTful services)
- Mobile Apps (with .NET MAUI/Xamarin)
- Cloud Services (Azure-based apps)
- Games (Unity engine uses C#)
- IoT and Embedded Systems



Basic C# Syntax Example

using System;

class Program

```
{  
    static void Main()  
    {  
        Console.WriteLine("Hello, BCA!");  
    }  
}
```

Popular C# Features

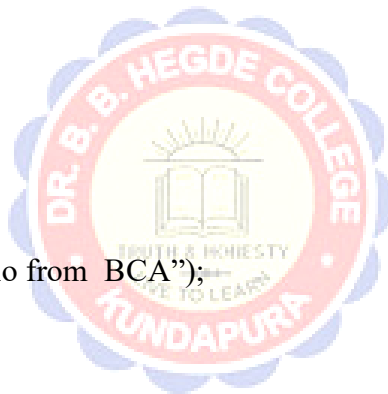
- Properties and Auto-Properties
- LINQ (Language Integrated Query)
- Delegates and Events
- Async/Await for asynchronous programming
- Exception Handling
- Generics
- Pattern Matching
- Nullable Types
- Attributes and Reflection

language features

C# is a **type safe object oriented language** that enables developers to build variety of secure and robust applications to execute on the .NET framework. User can create Windows client applications, XML web services, distributed components, client – server applications and database applications.

Simple C# Program

```
class SampleOne
{
    public static void Main()
    {
        System.Console.WriteLine("Hello from BCA");
    }
}
```



Class Declaration

The first line class SampleOne declares a class, which is an object oriented construct. Here, class is a keyword and declares a new class definition. SampleOne is a C# identifier that specifies the name of the class to be defined. A class is a template for what an object looks like and how it behaves.

The Braces

C# is a block structured language i.e. code blocks are always enclosed within { and }. Every class definition is enclosed in braces.

The Main Method

The third line public static void Main() defines a method named Main. Every C# executable program must include the Main method in one of the classes. A C# application can have any number of classes but only one class can have Main method to initiate the execution.

The meanings of the keywords that appear in the statement are

- public – It is an access modifier that tells C# compiler that the main method is accessible to everyone

- static – It declares that Main method is global and it can be called without creating an instance of the class. The compiler stores the address of the method as the entry point and uses this information to begin execution before any objects are created
- void – The keyword void is a type modifier that the main method does not return any value

The Output Line:

The only executable statement in C# is

```
System.Console.WriteLine("Hello from BCA Department");
```

C# is an object oriented language. Hence every method should be a part of an object. The WriteLine method is a static method of Console class, which is located in the namespace System. This line prints the string Hello from BCA Department to the screen. The WriteLine method always appends a new-line character to the end of the string. Every C# statement should end with a semi-colon.

Namespaces:

A class in a namespace can be accessed using the dot operator. Namespaces are the way C# segregates the .NET library classes into reasonable groups. Using directive issued to import the namespace into the program. Once the namespace is imported, we can use elements of that namespace without using namespace as prefix.

Example: using System;

```
class SampleOne
```

```
{  
    public static void Main()  
    {  
        Console.WriteLine("Hello from Preethi");  
    }  
}
```



Comments in C#

Comments are used to enhance the readability and understanding of code. C# supports two types of comments

- Single line comments – single line comments begin with //
- Multi-line comments – multi-line comments are enclosed within /* and */

Using Aliases for Namespaces

The using keyword can only be applied to namespaces and cannot be applied to classes. Hence, the statement

```
using System.Console ;
```

is illegal. We can overcome this by using aliases for namespace class. This takes the following syntax
using alias-name = class-name;

Example:

```
Using A = System.Console;
```

```
Class Sample1
```

```
{  
    public static void Main()  
    {  
        A.WriteLine("Hi I am Darshan III BCA, My best friend is Joshva");  
    }  
}
```

Passing strings as objects to WriteLine Method:

We can store string values in string objects and use these objects to the WriteLine method. We can create a string data type to create a string variable and assign a string constant to it as below:
 string s = "abc";

The content of s can be printed using the WriteLine method.

```
System.Console.WriteLine(s);
```

Example:

```
using System;
class Sample2
{
    public static void Main()
    {
        string name = "Hello Sona";
        Console.WriteLine(name);
    }
}
```

Structure of C# Program

Documentation Section
Using Directive Section
Interfaces Section
Classes Section
Main Method (compulsory)

Example:

```
/* Program to show a simple message */
using System;
namespace HelloWorldApplication
{
    class HelloWorld
    {
        static void Main(string[] args)
        {
            /* my first program in C# */
            Console.WriteLine("Hello Harshith");
        }
    }
}
```

Output:

Hello Harshith

Explanation:

- The first line of the program using System; - the using keyword is used to include the System namespace in the program. A program generally has multiple using statements.

- The next line has the namespace declaration. A namespace is a collection of classes. The HelloWorldApplication namespace contains the class HelloWorld.
- The next line has a class declaration, the class HelloWorld contains the data and method definitions that your program uses. Classes generally would contain more than one method. Methods define the behavior of the class. However, the HelloWorld class has only one method Main.
- The next line defines the Main method, which is the entry point for all C# programs. The Main method states what the class will do when executed
- The next line `/*...*/` will be ignored by the compiler and it has been put to add additional comments in the program.
- The Main method specifies its behavior with the statement `Console.WriteLine("Hello Harshith");` WriteLine is a method of the Console class defined in the System namespace. This statement causes the message "Hello, Harshith" to be displayed on the screen.

Assignment

2 marks

1. Define namespaces
2. How do you write comments in C#?
3. How can you use alias for namespaces?

5marks Questions

4. Explain the structure of C# program with an example

CHAPTER 2

LITERALS, VARIABLES, DATA TYPES

13.1 Introduction:

A C# program is basically a collection of classes. A class is defined by a set of declaration statements and methods containing instructions called as executable statements. These instructions are formed using certain symbols and words according to rigid rules known as syntax rules. The smallest, non-reducible element in a program is called as a **token**. C# program is a collection of tokens, comments and whitespaces. The five types of tokens in C# are

- Keywords
- Identifiers
- Literals
- Operators
- Punctuators

13.1.1 Keywords:

Keywords are reserved words predefined to the C# compiler. These keywords cannot be used as identifiers; however, if you want to use these keywords as identifiers, you may prefix the keyword with the `@` character. In C#, some identifiers have special meaning in context of code, such as `get` and `set`, these are called contextual keywords.

The following table lists the reserved keywords and contextual keywords in C#:

Reserved Keywords						
Abstract	As	Base	Bool	Break	Byte	case
Catch	Char	Checked	Class	Const	continue	decimal
Default	Delegate	Do	Double	Else	enum	event
Explicit	Extern	False	Finally	Fixed	float	for
Foreach	Goto	If	Implicit	In	in (generic modifier)	int
Interface	Internal	Is	Lock	Long	namespace	new

Null	Object	Operator	Out	out (generic modifier)	override	params
Private	Protected	Public	Readonly	Ref	return	sbyte
Sealed	Short	Sizeof	Stackalloc	Static	string	struct
Switch	This	Throw	True	Try	typeof	uint
Ulong	Unchecked	Unsafe	Ushort	Using	virtual	void
Volatile	While					
Contextual Keywords						
Add	Alias	Ascending	Descending	Dynamic	From	get
Global	Group	Into	Join	Let	orderby	partial (type)
Partial (method)	Remove	Select	Set			

13.1.2 Identifiers:

An identifier is a name used to identify a class, variable, function, or any other user-defined item. The basic rules for naming classes in C# are as follows:

- A name must begin with a letter that could be followed by a sequence of letters, digits (0 - 9) or underscore. The first character in an identifier cannot be a digit.
- It must not contain any embedded space or symbol like ? - + ! @ # % ^ & * () [] { } . ; : " ' / and \. However, an underscore (_) can be used.
- It should not be a C# keyword.

13.1.3 Operators

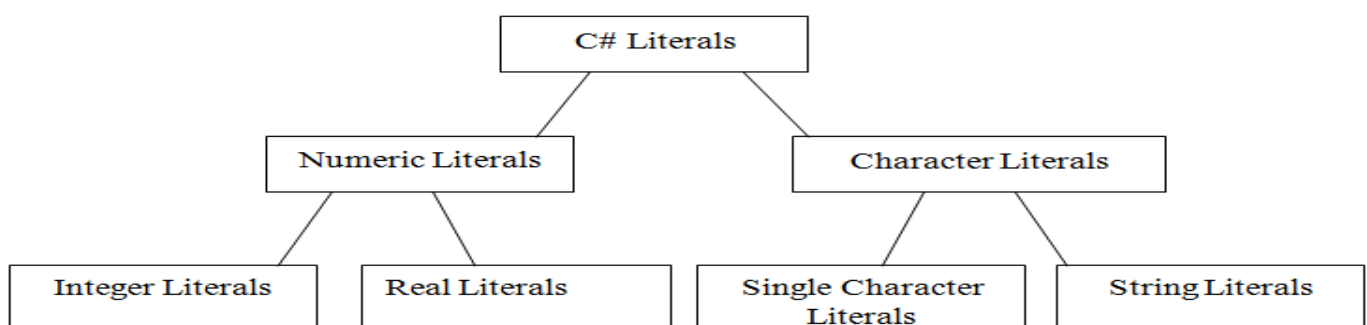
Operators are symbols used in expressions to describe operations involving one or more operands.

13.1.4 Punctuators:

Punctuators are symbols used to group and separate codes in C#. Punctuators are also known as separators. The punctuators in C# include

- Parentheses()
- Braces { }
- Brackets []
- Semi-colon ;
- Colon :
- Comma ,
- Period .

13.2 Literals:



13.2.1 Integer Literals:

An integer literal can be a decimal, octal, or hexadecimal constant. A prefix specifies the base or radix: 0x or 0X for hexadecimal, 0 for octal, and no prefix is required for decimal.

Examples:

```
212      /* Legal */
215u     /* Legal */
0xFeeL   /* Legal */
78       /* Illegal: 8 is not an octal digit */
```

13.2.2 Real Literals:

A floating-point literal has an integer part, a decimal point, a fractional part, and an exponent part. You can represent floating point literals either in decimal form or exponential form.

Here are some examples of floating-point literals:

```
3.14159   /* Legal */
314159E-5L /* Legal */
510E      /* Illegal: incomplete exponent */
210f      /* Illegal: no decimal or exponent */
.e55      /* Illegal: missing integer or fraction */
```

While representing using decimal form, you must include the decimal point, the exponent, or both and while representing using exponential form you must include the integer part, the fractional part, or both. The signed exponent is introduced by e or E.

13.2.3 Boolean Literals

There are two Boolean literals in C#. They are true and false. They are used as values for relational expressions.

13.2.4 Single Character Literals

A single character constant contains a single character enclosed within single quotes. Examples are 'x', '5', ' '.

13.2.5 String Literals

String literals or constants are enclosed in double quotes "". A string contains characters that are similar to character literals: plain characters, escape sequences, and universal characters. Examples : "hello", "hi", "Sooraj", "5+4"

13.2.6 Backslash constants:

There are certain characters in C# when they are preceded by a backslash they will have special meaning and they are used to represent like newline (\n) or tab (\t). Here, you have a list of some of such escape sequence codes:

Escape Sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits
\xhh . . .	Hexadecimal number of one or more digits

13.3 Variables:

A variable is an identifier that denotes a storage location used to store a data value. A variable may take different values at different times during the execution of a program. Every variable has a type that determines what values can be stored in a variable.

The rules to name a variable are

- They must not begin with a digit
- Uppercase and lower case letters are distinct
- It should not be a keyword
- Whitespaces are not allowed, We can begin variable name only with letter or underscore(_).
- Variable names can be of any length

13.4 Data Types

Data types specify the size and type of value to that can be stored. C# is a language rich in data types. The variety allows the programmer to select the type appropriate to the need of the application. The types in C# are

➤ Value types

➤ Reference types

Value types and reference types differ in

Where they are stored

How they behave in the context of assignment statements

Value types are of fixed length and they are stored on a stack, and when the value of the variable is assigned to another variable, the value is actually copied. This means that two identical copies of the values are available in the memory. Reference types are of variable length and are stored in a heap. When an assignment between two reference variables occurs, only the reference is copied and the actual value remains in the same memory location. This means that there are two references to a single variable. A third category called pointers is available for use only in unsafe codes

13.4.3 Value types

The value types in C# can be grouped into two categories

- ✓ User-defined types (Complex types)
 - ✓ Predefined types (Simple types)

We can define our own complex types known as user-defined value types which include **struct** type and **enumerations**. Predefined value types are also known as simple types or primitive types. They are further divided into

- Numeric types
- Boolean types
- Character types

The table below gives a list of predefined data types

Type	Represents	Range	Default Value
Bool	Boolean value	True or False	False
Byte	8-bit unsigned integer	0 to 255	0
Char	16-bit Unicode character	U +0000 to U +ffff	'\0'
Decimal	128-bit precise decimal values with 28-29 significant digits	$(-7.9 \times 10^{28} \text{ to } 7.9 \times 10^{28}) / 10^{0 \text{ to } 28}$	0.0M
Double	64-bit double-precision floating point type	$(+/-)5.0 \times 10^{-324} \text{ to } (+/-)1.7 \times 10^{308}$	0.0D
Float	32-bit single-precision floating point type	$-3.4 \times 10^{38} \text{ to } +3.4 \times 10^{38}$	0.0F
Int	32-bit signed integer type	-2,147,483,648 to 2,147,483,647	0
Long	64-bit signed integer type	-923,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
Sbyte	8-bit signed integer type	-128 to 127	0
Short	16-bit signed integer type	-32,768 to 32,767	0
UInt	32-bit unsigned integer type	0 to 4,294,967,295	0
Ulong	64-bit unsigned integer type	0 to 18,446,744,073,709,551,615	0
Ushort	16-bit unsigned integer type	0 to 65,535	0

13.4.4 Reference types

The reference types can also be divided into

- ✓ User-defined types
- ✓ Predefined types

User-defined reference types refer to those types which we define using predefined value types. They include

- Classes
- Interfaces
- Delegates
- Arrays

Predefined reference types include two data types:

- Object type
- String type

The object type is the ultimate base type of all other intrinsic and user-defined types in C#. We can use an object reference to bind an object of any particular type. C# creates its own string type for creating and manipulating strings.

13.5 Declaration of Variables:

A variable must be declared before it is used in the program. Declaration does three things!

1. It tells the compiler what the variable name is
2. It specifies what type of data the variable will hold
3. The place of declaration decides the scope of the variable

The syntax for variable declaration is

<data type> <variable list>;

Here, the data type must be a valid C# data type including char, int, float, double, or any user-defined data type, etc., and variable list may consist of one or more identifier names separated by commas.

Example:

```
int i, j, k; char c, ch;
float f, salary;
double d;
```

13.6 Initialization of Variables:

Variables are initialized (assigned a value) with an equal sign followed by a constant expression. The general form of initialization is:

variable_name = value;

Variables can be initialized (assigned an initial value) in their declaration. The initializer consists of an equal sign followed by a constant expression as:

<data_type> <variable_name> = value; Some examples are:

```
int d = 3, f = 5; /* initializing d and f. */
byte z = 22;      /* initializes z. */
double pi = 3.14159; /* declares an approximation of pi. */
char x = 'x';     /* the variable x has the value 'x'. */
```

13.7 Constant Variables:

Constants are defined using the **const** keyword. Syntax for defining a constant is:

const <data_type> <constant_name> = value;

The following program demonstrates defining and using a constant in your program:

```
using System;
namespace DeclaringConstants
{
    class Program
    {
        static void Main(string[] args)
        {
            const double pi = 3.14159; // constant declaration double r;
            Console.WriteLine("Enter Radius: ");
            r = Convert.ToDouble(Console.ReadLine());
            double areaCircle = pi * r * r;
            Console.WriteLine("Radius: {0}, Area: {1}", r, areaCircle);
        }
    }
}
```

```
        Console.ReadLine();  
    }  
}  
}
```

When the above code is compiled and executed, it produces the following result: Enter

Radius:

3

Radius: 3, Area: 28.27431

13.8 Scope of Variables:

The scope of a variable is the region of code within which the variable can be accessed. This depends on the type of the variable and place of its declaration. C# defines a several categories of variables. They are

- Static variables
- Instance variables
- Array elements
- Value parameters
- Reference parameters'
- Output parameters
- Local variables

13.9 Boxing and Unboxing:

In object-oriented programming, methods are invoked using objects. Since value types such as int and long are not objects, we cannot use them to call methods. C# enables us to achieve this through boxing. Boxing means the conversion of a value type on the stack to an object type on the heap. Conversely, the conversion from an object type back to a value type is known as unboxing.

13.9.3 Boxing :

Any type, value or reference can be assigned to an object without an explicit conversion. When the compiler finds a value type where it needs a reference type, it creates an object 'box' into which it places the value of the value type.

Example:

```
int i = 12;  
  
object box = i;  
if (box is int)  
{  
    Console.WriteLine("Box contains an int");  
}
```

13.9.4 Unboxing

Unboxing is the process of converting the object type back to value type. Unboxing is an explicit operation using C-style casting.

Example:

```
object box = 12; //boxing  
int i = (int)box; // unboxing
```

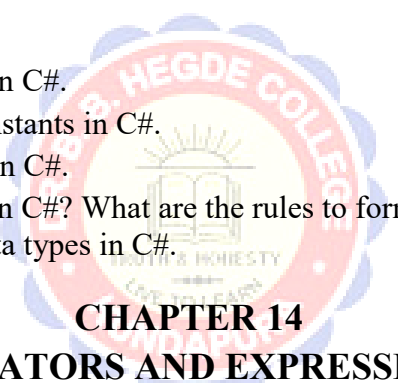
Assignment

2 mark Questions:

1. Define variable.
2. Define constant
3. What are trigraph characters? Give example.
4. What is a backslash constant? Give example.
5. Define keyword. Give example.
6. What is a token in C#? Give example
7. List the character set in C#.
8. How do you declare a variable in C#? Give example
9. How do you assign a value to a variable? Give example.
10. How do you declare a variable as a constant? Give example
11. How do you define a symbolic constant in C#? Give example
12. Differentiate between keyword and identifier
13. Define boxing and unboxing in C#

5 mark Questions:

1. Explain the primary data types in C#.
2. Explain the various types of constants in C#.
3. Explain about tokens available in C#.
4. How do you declare a variable in C#? What are the rules to form a variable?
5. Write a note on user defined data types in C#.



CHAPTER 14

OPERATORS AND EXPRESSIONS

14.1 Introduction:

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C# is rich in built-in operators and provides the following type of operators:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Increment/decrement operators
- Conditional operators
- Bitwise Operators
- Special Operators

Arithmetic Operators:

All the basic arithmetic operators are supported by C#. These operators can manipulate with any built in data types supported by C#. The various operators are tabulated as follows:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo Division

14.2.1 Integer Arithmetic:

When both the operands in the expression are integers, then the operation is known as integer arithmetic.

For example if the values of $a=14$ and $b=4$ then

$a+b=18$, $a-b=10$, $a*b=56$, $a/b=3$, $a\%b=2$

During modulo division, the sign of the result is the sign of the first operand. E.g. -

$14\%3=-2$, $14\%-3=2$

14.2.2 Real Arithmetic:

An arithmetic operation involving only real operands is called real arithmetic. A real value can assume value either in decimal or exponential notation. % operator cannot be used with real operands

E.g. If $a=2.4$ and $b=1.2$ then

$a+b=3.6$, $a-b=1.2$, $a*b=2.88$ $a/b=2.0$

14.2.3 Mixed mode Arithmetic:

When one of the operands is real and the other is an integer, the expression is called mixed mode arithmetic expression. The result is always a real number

E.g. : $15/10.0=1.5$, $15/10=1.5$

14.3 Relational Operators:

Comparisons can be done using relational operators. A relational expression contains a combination of arithmetic expressions, variables or constants along with relational operators. A relational expression can contain only two values i.e. true or false. When the expression is evaluated as true then the compiler assigns a non zero value and 0 otherwise. These expressions are used in decision statements to decide the course of action of a running program.

Syntax: $ae1$ relational operator $ae2$

where $ae1$ and $ae2$ are arithmetic expressions

Operators	Meaning
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equal to
!=	Not equal to

14.4 Logical Operators:

An expression that com

bines two or more relational expressions is called a logical expression and the operators used to combine them are called logical operators.

Syntax:

R1 operator R2

where, R1 and R2 are relational expressions

Operator	Meaning
&&	Logical And
	Logical Or
!	Logical Not

Relational Expression R1	Relational Expression R2	R1&&R2	R1 R2
0	0	0	0
0	Non zero	0	Non zero
Non zero	0	0	Non zero
Non zero	Non zero	Non zero	Non zero

14.5 Assignment Operators:

Assignment operators are used to assign the result of an expression to variable. The operator used for assignment in C# is '='. C# also supports short hand assignment operators like

v op = expression where v- variable and op- operator. Here op= is known as a short had assignment operator.

v op=expression is equivalent to v=v op expression E.g. The statement x+=y+1; is equivalent to x=x+y+1; The advantage of short hand assignment operator is Easy to read and write Statements are more concise and efficient

14.6 Increment/Decrement Operators:

C# has 2 very powerful operators that are not found in any other language. They are increment/decrement operators i.e. ++ and --. The ++ operator is used to increment value of a variable by 1. The -- operator is used to decrement value of a variable by 1. Both are unary operators and can be written as follows: ++m, m++ , m--and --m. Both m++ and ++m mean the same thing when they form statements independently. But, they behave differently when used in expression on the right hand side of assignment operator.

E.g. Let a=5; x=a++; Here, this statement can be broken into 3 statements as follows a=5; x=a;

a=a+1;

In the above example, the value of a is assigned to x and then its value is incremented by 1. A prefix operator first adds 1 to the operand and then the result is assigned to the variable on the left. A postfix operator first assigns its value to the variable and then increments its value by 1. E.g. m=10;

y=--m

Here the value of m is decremented by 1 and then assigned to y. Hence the value of y is 9.

14.7 Conditional Operator:

Conditional operators are also called ternary operators. Conditional expressions can be constructed in C# using the operator pair '?:'.

Syntax: `expr?expr1:expr2;`

Here `expr` is evaluated first. If the value is true, then `expr1` is evaluated and becomes the value of the expression. If the expression is false then, `expr2` is evaluated and becomes the value of the expression.

E.g. `a=5; b=10;`

`x=(a>b)?a:b;`

The output of the above example is as follows- The value of `b` i.e. 10 is assigned to `x`.

14.8 Bitwise Operators:

These operators are used to manipulate data at bit level. These operators are used for testing the bits or shifting the bits either to the left or right.

Operator	Meaning
<code>&</code>	Bitwise And
<code> </code>	Bitwise Or
<code>^</code>	Bitwise exclusive Or
<code><<</code>	Shift left
<code>>></code>	Shift Right
<code>~</code>	One's complement

The Bitwise operators supported by C# are listed in the following table. Assume variable `A` holds 60 and variable `B` holds 13 then:

Operator	Description	Example
<code>&</code>	Binary AND Operator copies a bit to the result if it exists in both operands.	(<code>A & B</code>) will give 12, which is 0000 1100
<code> </code>	Binary OR Operator copies a bit if it exists in either operand.	(<code>A B</code>) will give 61, which is 0011 1101
<code>^</code>	Binary XOR Operator copies the bit if it is set in one operand but not both.	(<code>A ^ B</code>) will give 49, which is 0011 0001
<code>~</code>	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(<code>~A</code>) will give -61, which is 1100 0011 in 2's complement due to a signed binary number.
<code><<</code>	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	<code>A << 2</code> will give 240, which is 1111 0000
<code>>></code>	Binary Right Shift Operator. The left	<code>A >> 2</code> will

moved right by the number of bits specified by the right operand.

give 15, which is 0000 1111.

14.9 Special Operators:

C# supports the following special operators.

Operator	Description	Example
sizeof()	Returns the size of a data type.	sizeof(int), will return 4.
typeof()	Returns the type of a class.	typeof(StreamReader);
&	Returns the address of an variable.	&a; will give actual address of the variable.
*	Pointer to a variable.	*a; will pointer to a variable.
? :	Conditional Expression	If Condition is true ? Then value X : Otherwise value Y
Is	Determines whether an object is of a certain type.	If(Ford is Car) // checks if Ford is an object of the Car class.
As	Cast without raising an exception if the cast fails.	Object obj = new StreamReader("Hello"); StreamReader r = obj as StreamReader;

14.10 Arithmetic Expressions:

An arithmetic expression is a combination of variables, constants and operators arranged as per the syntax of the language. Expressions are evaluated using assignment statements of the form

variable=expression;

Expressions are always evaluated from left to right, using the rules of precedence of operators, if parenthesis are missing

High Precedence - * / % Low

Precedence - + -

Basically the evaluation procedure involves 2 left to right passes. During the first pass the high priority operators are applied as they are encountered and during the second pass, lower priority operators are applied as they are encountered.

Example: Consider the following expression $x = a - b/3 + c*2 - 1$

If the values of $a=9$, $b=12$, $c=3$

Now $x = 9 - 12/3 + 3*2 - 1$

Pass 1: $x = 9 - 4 + 6 - 1$

Pass 2: $x = 5 + 6 - 1 = 10$

14.11 Precedence of Arithmetic Operators:

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator:

For example $x = 7 + 3 * 2$; here, x is assigned 13, not 20 because operator $*$ has higher precedence than $+$, so it first gets multiplied with $3*2$ and then adds into 7.

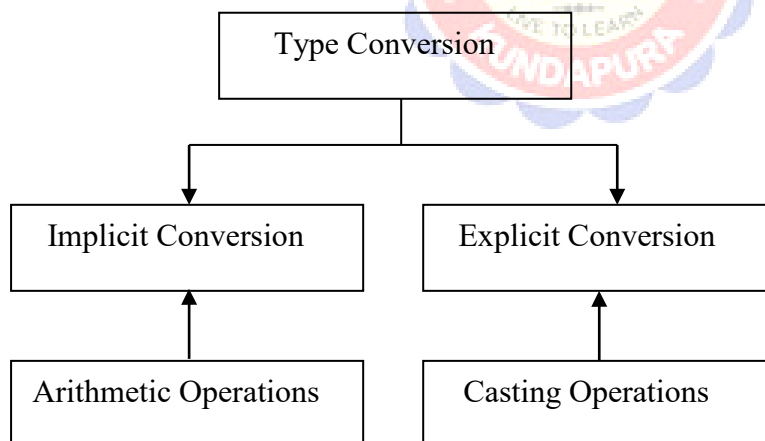
Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

14.12 Type Conversions:

Type conversion is basically type casting or converting one type of data to another type. In C#, type casting has two forms:

- **Implicit type conversion** - these conversions are performed by C# in a type-safe manner. Examples are conversions from smaller to larger integral types and conversions from derived classes to base classes. The process of assigning a smaller type to a larger one is called *widening* or *promotion*. The process of assigning a larger type to a smaller one is called *narrowing*.
- **Explicit type conversion** - these conversions are done explicitly by users using the pre-defined functions. Explicit conversions require a cast operator.



Example for Explicit Type conversion:
namespace TypeConversionApplication

```

{
    class ExplicitConversion
    {
        static void Main(string[] args)
        {
            double d = 5673.74;
            int i;

            // cast double to int. i = (int)d;
        }
    }
}
  
```

```

        Console.WriteLine(i);
        Console.ReadKey();
    }
}
}

```

14.13 Mathematical Functions:

The System namespace defines a class called Math class with a rich set of static methods that makes math-oriented programming easy and efficient. The following table lists some of the mathematical methods defined in Math class.

Method	Meaning
Ceil()	x rounded up to the nearest integer
Sin()	sine of the angle in radians
Log10()	Base 10 logarithm
Floor()	x rounded down to the nearest integer
Log()	Natural log of x, $x > 0$
Pow(X,Y)	x raised to y (x^y)
Sqrt(X)	Square root of x ; $x \geq 0$

Example: using System;

class Program

```

{
    static int Main()
    {
        int number = -6844;

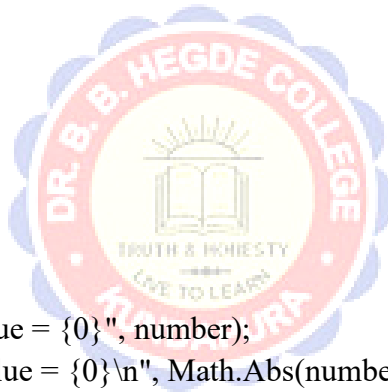
        Console.WriteLine("Original Value = {0}", number);
        Console.WriteLine("Absolute Value = {0}\n", Math.Abs(number)); return 0;
    }
}

```

This would produce: Original

Value = -6844

Absolute Value = 6844



2 mark questions

1. What is the difference between pre-increment and post-increment operators
2. If the value of $a=5$ then, explain $a--$ and $--a$.
3. What is short hand assignment in C#? Give example
4. What is the purpose of ternary operator? Give example
5. What do you mean by typecasting? Give example
6. List any four mathematical functions in C#.
7. What is the purpose of size of operator?
8. Why are bitwise operators used in C#?
9. If $a=10$, $b=20$ evaluate the following statements in C#
 - a. $a+=10$;
 - b. $a+4/6*6/2$; c. $b+3/2*2/3$; d. $a+=b$;
 - e. $c=a++$;

f. `d--b;`

5 Marks Questions

1. Explain about arithmetic operators.
2. Write a note on bit-wise operators.
3. Explain about logical operators
4. Write a note on relational operators.
5. Explain about increment operators in C#.
6. Explain about decrement operators in C#.

Short Questions for 2 marks

1. Differentiate between value type and reference types.
2. How do you define constants variables in C#?
3. What does public static void means in the declaration of main method ?

```
public static void main()  
{.....}
```

4. What are Command line arguments? Give example.
5. What is an alias? Where and how it is used?

6. How do we implement a program having more than one class with the Main method? (multiple main methods)
7. List the special operators used in C# ,specify their purpose.
8. What is Mixed-mode arithmetic?
9. What is SOAP ?
10. What is Thread ? Write the different Thread states.
11. Write any two properties of a Thread.

Long answer questions

1. How does C# differ from Java ?
2. Explain the different types of programs developed using C# (Executable , Libraries).
3. Explain the different Data Types (value and reference type with taxonomy of C# data type fig).
4. Explain the different Value types used in C#.
5. What is Literal? Explain.
6. What are command line arguments ? Explain with example.
7. Explain Scope of variable with example.
8. What is Operator ? List the different types , explain any 4 types.
9. Explain Implicit conversion with example.
10. Explain Explicit conversion with example.
11. Explain the different elements of web service.
12. Explain the different states of ASP.NET.
13. Write a note on UDDI.
14. Explain WSDL.
15. Write a note on Deployment .
16. What is Caching ? Explain the different types.
17. Explain the security mechanism used in ASP.NET.

CHAPTER 15

DECISION MAKING AND BRANCHING

15.1 Introduction:

The program statements are executed sequentially. But, sometimes we need to change the order of execution of statements depending on certain conditions. C# language possesses decision making capabilities and supports statements called decision making statements. The decision making statements supported by C# are as follows:

1. if statement
2. switch statement
3. Conditional operator statement

15.2 The if statement:

The if statement is a powerful decision making statement and is used to control the flow of execution of statements. It is basically a two-way decision statement used in conjunction with an expression. The computer evaluates the value of the expression first and depending on the value of the expression it transfers control to a particular statement. The if statement can be implemented in different forms depending on the complexity of the conditions to be tested. The different forms of if statements are:

- Simple if
- If...else statement
- Nested if...else statement
- Elseif ladder

15.2.1 Simple if statement:

The general form of simple if statement is as follows if(test (expression)

```

{
    statement block;
}
statement x;
```



The statement block may be a single statement or a group of statements. If the test expression is true, the statement block will be executed; otherwise the control jumps to statement x. However, if the condition is true both the statement block as well as statement x are executed in a sequence.

15.2.2 if..else statement:

The if else statement is an extension of the simple if statement. The general format of it is as follows:

```

if(test-expression)
{
    True Block-Statement;
}
Else
{
    False-block statements;
}
Statement x;
```

If the test expression is true, then the true block statements are executed otherwise the false block statements are executed. Either of the two statements, i.e. true block or false block statements only are executed, but not both. In both the cases, the control transfers to statement x after executing the block.

Example

using System;

namespace DecisionMaking

{

class Program

{

static void Main(string[] args)

{

/* local variable definition */

int a = 100;

/* check the boolean condition */

if (a < 20)

{

/* if condition is true then print the following */

Console.WriteLine("a is less than 20");

}

else

{

/* if condition is false then print the following */

Console.WriteLine("a is not less than 20");

}

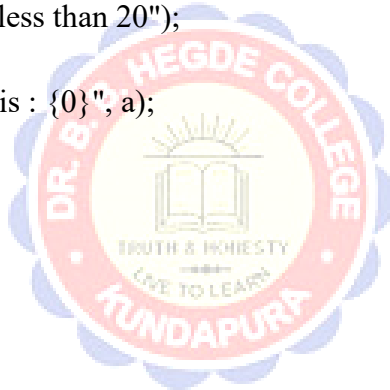
Console.WriteLine("value of a is : {0}", a);

Console.ReadLine();

}

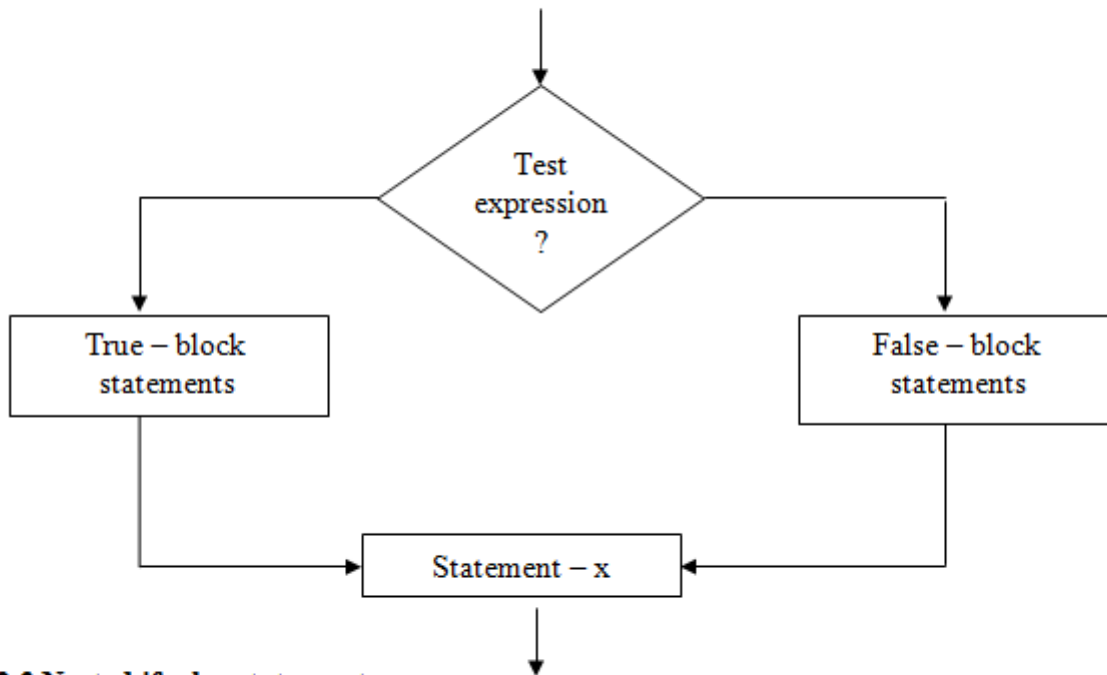
}

}



When the above code is compiled and executed, it produces the following result:

a is not less than 20;
value of a is : 100



15.2.3 Nested if..else statement:

When a series of decisions are involved, we may have to use more than one if..else statement one within the other. This form of if..else statement is known as nested if..else statement. The general format of this statement is as follows:

```

if(test condition1)
{
    if (test condition2)
        statement 1;
    else
        statement 2;
}
else
    statement 3;
statement x;
  
```



If condition 1 is initially false, statement 3 will be executed; otherwise it continues to evaluate condition 2. If condition 2 is true then statement 1 is executed; otherwise statement 2 will be executed and then the control transfers to statement x.

Example

using System;

namespace DecisionMaking

{

class Program

{

static void Main(string[] args)

{

```

    /** local variable definition */
    int a = 100;
    int b = 200;

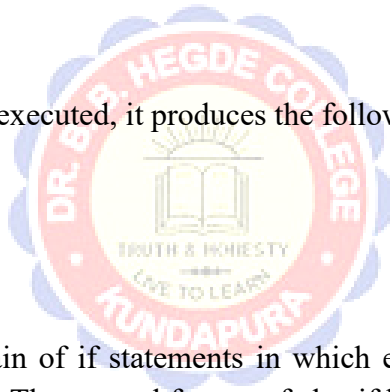
    /* check the boolean condition */
    if (a == 100)
    {
        /* if condition is true then check the following */
        if (b == 200)
        {
            /* if condition is true then print the following */
            Console.WriteLine("Value of a is 100 and b is 200");
        }
    }
    Console.WriteLine("Exact value of a is : {0}", a);
    Console.WriteLine("Exact value of b is : {0}", b);
    Console.ReadLine();
}
}
}

```

When the above code is compiled and executed, it produces the following result: Value of a is 100 and b is 200

Exact value of a is : 100

Exact value of b is : 200



15.2.4 The Else-if Ladder:

A multi path decision is a chain of if statements in which each else is an if statement. This construct is known as an else-if ladder. The general format of else-if ladder is as follows:

```

    if(condition 1)
        statement 1;
    elseif(condition 2)
        statement 2;
    elseif(condition 3)
        statement 3;
    :
    :
    elseif(condition n)
        statement n;
    else
        default statement;
    statement x;

```

Here the conditions are evaluated from top. As soon as a true condition is encountered, the statements associated with it are executed and the control is transferred to statement x. If all the conditions are false, then the final else containing the default statement will be executed.

Example:

using System;

namespace DecisionMaking

{

class Program

{

static void Main(string[] args)

{

/* local variable definition */

int a = 100;

/* check the boolean condition */

if (a == 10)

{

/* if condition is true then print the following */

Console.WriteLine("Value of a is 10");

}

else if (a == 20)

{

/* if else if condition is true */

Console.WriteLine("Value of a is 20");

}

else if (a == 30)

{

/* if else if condition is true */

Console.WriteLine("Value of a is 30");

}

else

{

/* if none of the conditions is true */ Console.WriteLine("None of the values is matching");

}

Console.WriteLine("Exact value of a is: {0}", a);

Console.ReadLine();

}

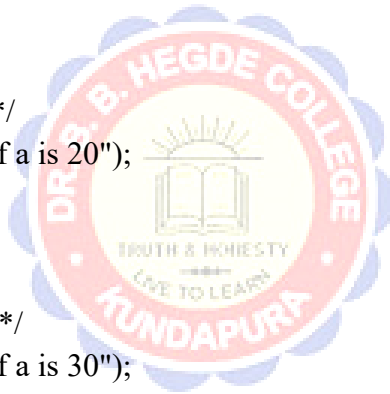
}

}

When the above code is compiled and executed, it produces the following result:

None of the values is matching

Exact value of a is: 100



15.3 The switch statement:

C# language supports a built-in multi way decision statement known as switch statement. The switch statement tests the value of a given variable against a list of case values and when a match is found, the block of statements associated with that case are executed. The general form of switch statement is as follows:

```
switch(expression){
    case constant-expression :
statement(s);

    break; /* optional */
case constant-expression :
    statement(s);
    break; /* optional */
/* you can have any number of case statements */
default : /* Optional */
    statement(s);
}
```

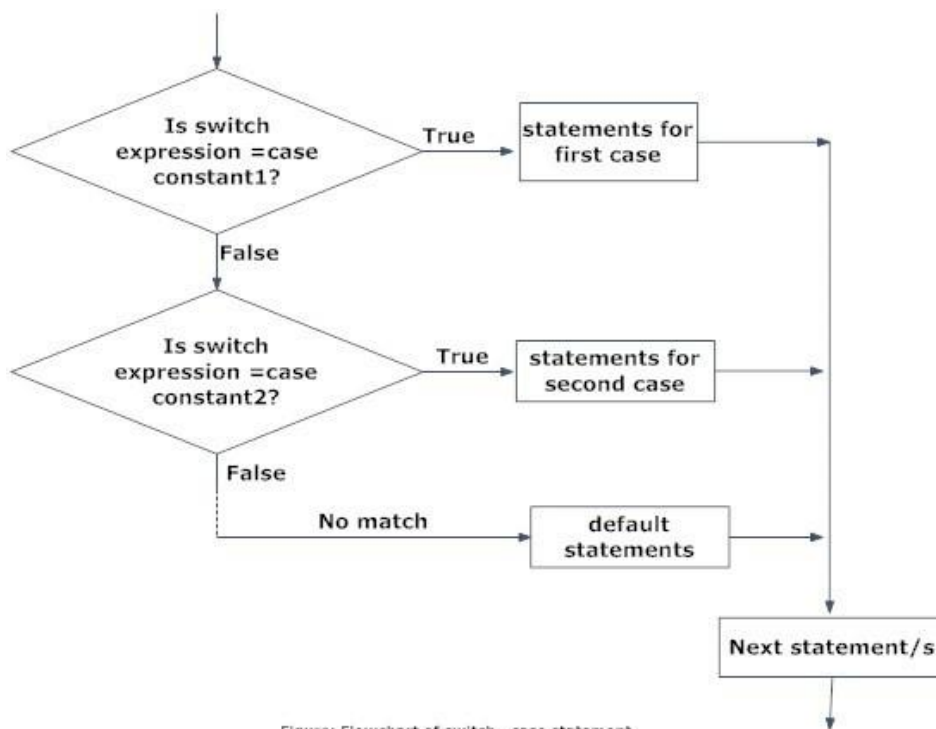
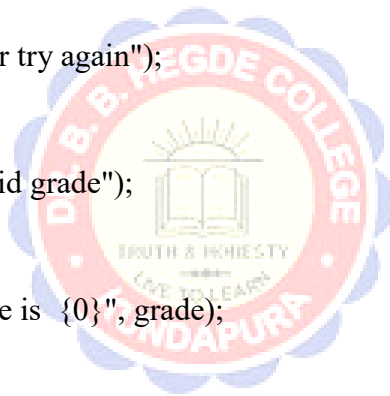


Figure: Flowchart of switch...case statement

The expression can either be an integer expression or a character expression. Value1, value 2,...value n are constants known as case labels. Each of these values should be unique within a switch statement. Case labels must end with a colon(:). When the switch statement is executed, the value of the expression is successively compared against the values value1, value 2,...value n. If a match is found, the block of statements that follow the case are executed. The break statement in each block signifies the end of that case and causes exit from the switch statement. The default case is an optional one. The statements in this case are executed when no case labels match with the value of that expression.

Example:

```
using System;
namespace DecisionMaking
{
    class Program
    {
        static void Main(string[] args)
        {
            char grade = 'B'; /* local variable definition */
            switch (grade)
            {
                case 'A': Console.WriteLine("Excellent!");
                    break;
                case 'B':
                case 'C':
                    Console.WriteLine("Well done");
                    break;
                case 'D':
                    Console.WriteLine("You passed");
                    break;
                case 'F':
                    Console.WriteLine("Better try again");
                    break;
                default:
                    Console.WriteLine("Invalid grade");
                    break;
            }
            Console.WriteLine("Your grade is {0}", grade);
            Console.ReadLine();
        }
    }
}
```



When the above code is compiled and executed, it produces the following result: Well done
Your grade is B

Statement	Description
if statement	An if statement consists of a boolean expression followed by one or more statements.
if...else statement	An if statement can be followed by an optional else statement, which executes when the boolean expression is false.
nested if statements	You can use one if or else if statement inside another if or else if statement(s).
switch statement	A switch statement allows a variable to be tested for equality against a list of values.

2 mark Questions:

1. Give the syntax of simple if with example.
2. What is the purpose of goto statement?
3. What is ternary operator? Give example.
4. Give the syntax of switch statement.

5 mark Questions:

1. Explain the various forms if statement with example.
2. Explain the working of switch statement with an example.

CHAPTER 16

DECISION MAKING AND LOOPING

16.1 Introduction

Looping is executing a sequence of statements until some conditions to terminate the loop are satisfied. A program loop consists of 2 segments i.e. body of the loop and control statements. The control statements test certain condition and then direct repeated execution of statements contained in the body of the loop. A looping process includes the following steps

1. Setting and initializing of a counter
2. Execution of statements in a loop
3. Test for specified condition for executing the loop
4. Incrementing the counter

C# supports 3 loop constructs to perform looping operations. They are as follows

- While statement
- Do...while statement
- For statement

16.2 The while statement:

The while statement is an entry-controlled loop. Here, the test condition is evaluated first and if the condition is true the body of the loop is executed. After executing the statements contained in the body of the loop, the test condition is again evaluated to check if it is true. This process is repeated until the test condition becomes false; and the control is transferred out of the loop. The general form of while statement is

```
initialization;
while(test condition)
{
    Body of the loop;
}
```

Here, the body of the loop can contain more than one statement. If the condition is initially false, the body of the loop will not be executed at all.

Example:

using System;

```
namespace Loops
{
    class Program
    {
```

```
static void Main(string[] args)
{
    /* local variable definition */
    int a = 10;

    /* while loop execution */
    while (a < 20)
    {
        Console.WriteLine("value of a: {0}", a);
        a++;
    }
    Console.ReadLine();
}
```

When the above code is compiled and executed, it produces the following result:

value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19

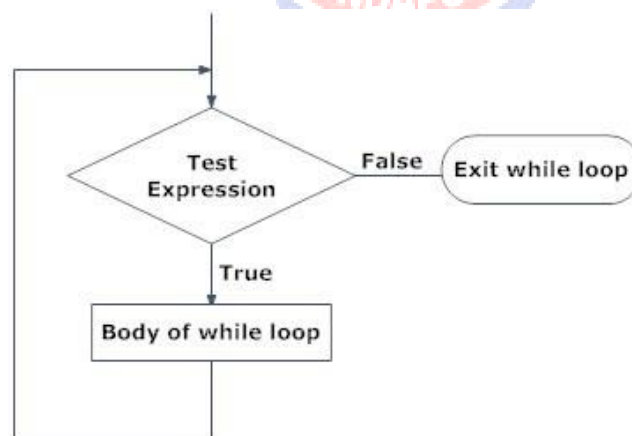


Figure: Flowchart of while loop

16.3 The do...while statement:

This loop is an exit controlled loop because the condition is checked after executing the statements in the loop. Here the statements are executed first and then, checks the condition. The body of the loop is executed at least once even if the condition is initially false. On reaching the do statement the body of the loop is executed. The test condition in the while statement is evaluated. If the condition is true, execution of the loop is repeated. This process continues as long as the condition specified in the while statement is true. Once the condition becomes false, the loop is terminated and the control goes to the statement after the

while statement.

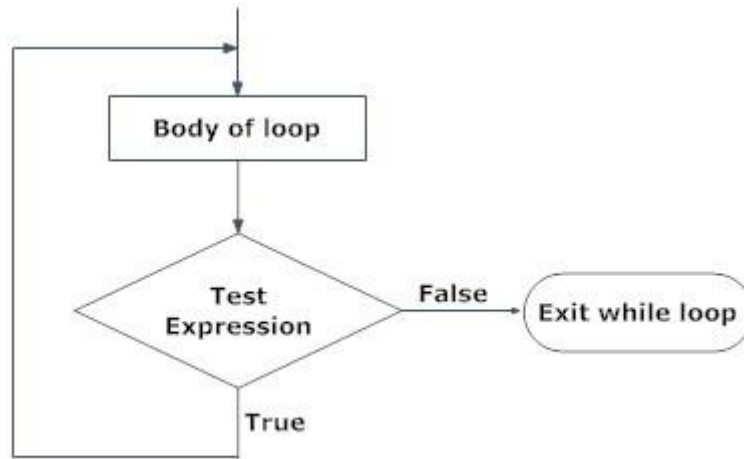


Figure: Flowchart of do...while loop

The general form of do while statement is as follows:

```

do
{
statements;
}
while (condition);
  
```

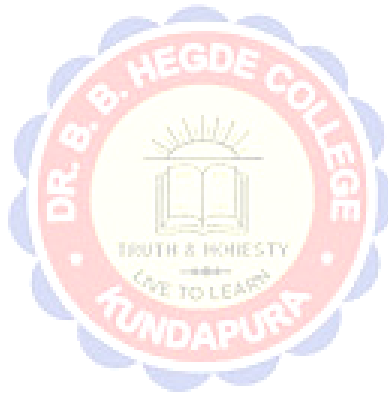
Example: using System;
namespace Loops

```

{
class Program
{
static void Main(string[] args)
{
/* local variable definition */
int a = 10;

/* do loop execution */
do
{
Console.WriteLine("value of a: {0}", a);
a = a + 1;
} while (a < 15);

Console.ReadLine();
}
}
}
  
```



When the above code is compiled and executed, it produces the following result:

```

value of a: 10 value of a:
11 value of a: 12 value
of a: 13 value of a: 14
  
```


16.4 For statement:

This statement is also an entry-controlled loop that provides a concise loop control structure. The general form of for loop is as follows:

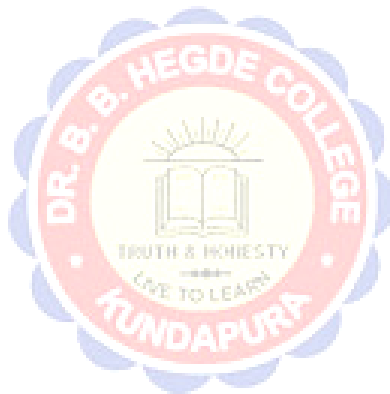
```
for(initialization; test condition; increment/decrement)
{
Body of the loop;
}
```

The execution of this loop is as follows:

- Initialization of the control variable is done using the assignment operator. More than one assignment can be made here, but, they must be separated by a comma operator.
- The value of the control variable is tested in the test condition. The test condition is any relational expression that determines when the loop will be terminated. The body of the loop executes as long as the condition is true.
- After the body of the loop is executed the control is transferred back to the for statement. Now the control variable is either incremented or decremented and its new value is tested to check if it satisfies the loop condition. This process continues till the value of the control variable fails to satisfy the test condition

Example:

```
using System;
namespace Loops
{
    class Program
    {
        static void Main(string[] args)
        {
            for(i=0; i<=4; i++)
            {
                Console.WriteLine(i);
            }
            Console.ReadLine();
        }
    }
}
```



When the above code is compiled and executed, it produces the following result:

```
0
1
2
3
4
```

16.5 For Each statement:

The foreach statement repeats a group of embedded statements for each element in an array or an object collection that implements the `System.Collections.IEnumerable` or `System.Collections.Generic.IEnumerable<T>` interface. The foreach statement is used to iterate through the collection to get the information that you want, but cannot be used to add or remove items from the source collection to avoid unpredictable side effects. If you need to add or remove items from the source

collection, use a for loop.

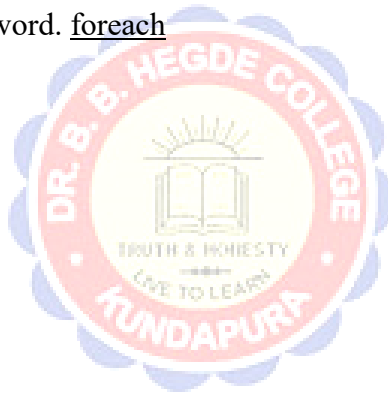
The embedded statements continue to execute for each element in the array or collection. After the iteration has been completed for all the elements in the collection, control is transferred to the next statement following the foreach block. At any point within the foreach block, you can break out of the loop by using the break keyword, or step to the next iteration in the loop by using the continue keyword. A foreach loop can also be exited by the goto, return, or throw statements.

Example:

```
using System;
class Program
{
    static void Main()
    {
        // Use a string array to loop over. string[]
        flowers =
        {
            "Rose", "Lily",
            "Lotus", "Tulip"
        };
        // Loop with the foreach keyword. foreach
        (string value in flowers)
        {
            Console.WriteLine(value);
        }
    }
}
```

Output:

Rose, Lily,
Lotus,
Tulip



Loop Type	Description
while loop	Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
for loop	Execute a sequence of statements multiple times and abbreviates the code that manages the loop variable.
do...while loop	Like a while statement, except that it tests the condition at the end of the loop body
nested loops	You can use one or more loop inside any another while, for or do..while loop.

16.6 Break statement:

In order to exit out of a loop, we use the break statement. When a break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement following the loop. When a break statement is encountered in a nested loop then, it will only exit from the loop containing the break statement i.e. a break statement is used to exit from a single loop only.

Syntax:

```
break;
```

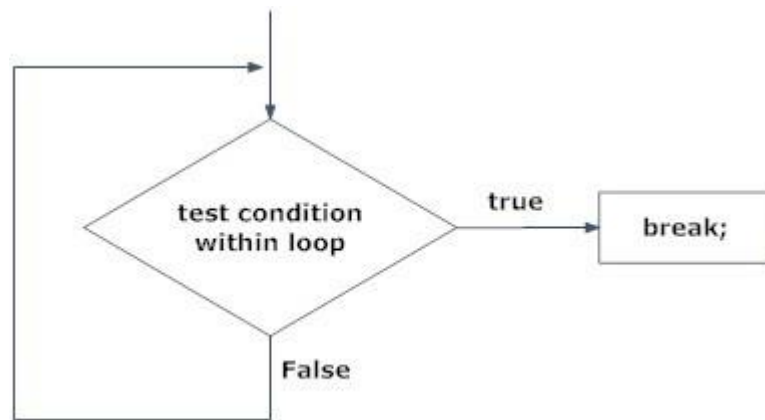


Figure: Flowchart of break statement

16.7 Continue statement:

The continue statement causes the loop to be continued with the next iteration after skipping any statements. The continue statement tells the compiler to skip the following statements and continue with the next iteration.

Syntax:

```
continue;
```

Whenever the continue statement is encountered, the other statements are ignored by the compiler and the control directly passes to the test condition to execute the next iteration. In case of for loop, the increment section is executed before the test condition is evaluated.

The **difference** between break and continue statements is that the break statement causes the loop to be terminated whereas the continue statement continues to execute the loop with the next iteration.

Assignment

2 mark Questions:

1. What is the purpose of break statement in C#?
2. What is the purpose of continue statement in C#?
3. Give the difference between break and continue statements.
4. Give the syntax of while loop.
5. Give the syntax of do..while loop.
6. Give the syntax of for loop.
7. Give the syntax of for-each loop.

5 mark Questions:

1. Explain the working of while loop with syntax and example
2. Explain the working of do..while loop with syntax and example
3. Explain the working of for loop with syntax and example
4. Explain the working of for each loop with syntax and example

CHAPTER 17

METHODS IN C#

17.1 Introduction

In object oriented programming, objects are used as building blocks in developing a program. They are runtime entities. They may represent a person, place, bank account or any item that the program can handle. Objects encapsulate data and code to manipulate that data. The code designed to work on data is called **methods** in C#. Methods give objects their behavioral characteristics. They are used not only to access and process data contained in the object but also to provide responses to any messages received from other objects.

17.2 Declaring Methods:

Methods are declared inside the body of the class, normally after the declaration of the data fields. The general form of a method declaration is

```
<Access Specifier> <Return Type> <Method Name>(Parameter List)
{
    Method Body
}
```

Following are the various elements of a method:

- **Access Specifier:** This determines the visibility of a variable or a method from another class.

Following code snippet shows a function FindMax that takes two integer values and returns the larger of the two. It has public access specifier, so it can be accessed from outside the class using an instance of the class.

```
class NumberManipulator
{
    public int FindMax(int num1, int num2)
    {
        /* local variable declaration */
        int result;

        if (num1 > num2)
            result = num1;
        else
            result = num2;
        return result; } }
```

17.3 Invoking Methods

The process of activating a method is known as calling or invoking a method. The invoking is done using the dot(.) operator.

The general format is

objectname.method name(actual-parameter list);

In the parameter list, the actual parameters are separated by commas that must match in type, order and number with the formal parameter list of the method name declared in the class.

Example:

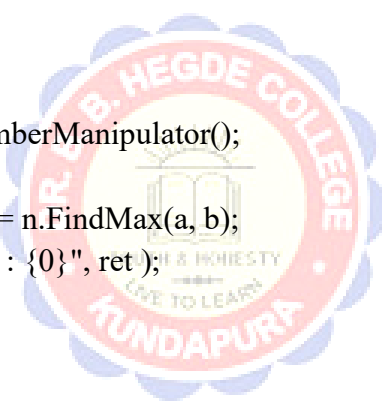
```
using System;
namespace CalculatorApplication
{
    class NumberManipulator
    {
        public int FindMax(int num1, int num2)
        {
            /* local variable declaration */
            int result;

            if (num1 > num2)
                result = num1;
            else
                result = num2;

            return result;
        }
        static void Main(string[] args)
        {
            /* local variable definition */
            int a = 100; int b =
            200; int ret;
            NumberManipulator n = new NumberManipulator();

            //calling the FindMax method ret = n.FindMax(a, b);
            Console.WriteLine("Max value is : {0}", ret);
            Console.ReadLine();

        }
    }
}
```



When the above code is compiled and executed, it produces the following result: Max value is : 200

17.4 Nesting of Methods:

A method can be called using only its name by another method of the same class. This is called nesting of methods.

Example:

```
using System;
class Nesting
{
    void Largest(int m, int n)
    {
        int large=Max(m,n);
        Console.WriteLine(large);
    }
    int Max(int a, int b)
```

```
{
    int x= (a>b)?a:b;
    return(x);
}
}
class NestTest
{
    public static void main()
    {
        Nesting.next = new Nesting();
        next.Largest(100,200);

    }
}
```

17.5 Method Parameters:

The different kinds of parameters are

Value Parameters

- Reference parameters
- Output parameters
 - Parameter arrays

17.5.1 Value parameters:

Value parameter is also called In parameter. A parameter declared with no modifiers is a value parameter. A value parameter corresponds to a local variable that gets its initial value from the corresponding argument supplied in the method invocation. A method is permitted to assign new values to a value parameter. Such assignments only affect the local storage location represented by the value parameter- they have no effect on the actual argument given in the method invocation.

Example:

using System;

namespace CalculatorApplication

```
{
    class NumberManipulator
    {
        public void swap(int x, int y)
        {
            int temp;

            temp = x; /* save the value of x */
            x = y; /* put y into x */
            y = temp; /* put temp into y */
        }

        static void Main(string[] args)
        {
            NumberManipulator n = new NumberManipulator();
```

```

        /* local variable definition */
        int a = 100;
int b = 200;

Console.WriteLine("Before swap, value of a : {0}", a);

Console.WriteLine("Before swap, value of b : {0}", b);

        /* calling a function to swap the values */
        n.swap(a, b);

        Console.WriteLine("After swap, value of a : {0}", a);
        Console.WriteLine("After swap, value of b : {0}", b);

        Console.ReadLine();
    }
}

```

When the above code is compiled and executed, it produces the following result: Before swap, value of a :100
Before swap, value of b :200
After swap, value of a :100
After swap, value of b :200

17.5.2 Reference parameters:

A parameter declared with a ref modifier is a reference parameter. Contrastingly to value parameter, a reference parameter does not make a new storage location. Instead, a reference parameter represents the same storage location as the variable given as the argument in the method invocation. Within a method, a reference parameter is always considered definitely assigned.

Example:

```

using System;
namespace CalculatorApplication
{
    class NumberManipulator
    {
        public void swap(ref int x, ref int y)
        {
            int temp;

            temp = x; /* save the value of x */
            x = y; /* put y into x */
            y = temp; /* put temp into y */
        }

        static void Main(string[] args)
        {
            NumberManipulator n = new NumberManipulator();
            /* local variable definition */

```

```
int a = 100;
int b = 200;

Console.WriteLine("Before swap, value of a : {0}", a);
Console.WriteLine("Before swap, value of b : {0}", b);

/* calling a function to swap the values */
n.swap(ref a, ref b);

Console.WriteLine("After swap, value of a : {0}", a);
Console.WriteLine("After swap, value of b : {0}", b);

Console.ReadLine();

    }
}
}
```

When the above code is compiled and executed, it produces the following result: Before swap, value of a : 100
Before swap, value of b : 200
After swap, value of a : 200
After swap, value of b : 100

17.5.3 Output parameters:

A parameter declared with an out modifier is an output parameter. Similar to a reference parameter, an output parameter does not create a new storage location. Instead, an output parameter represents the same storage location as the variable given as the argument in the method invocation. Every output parameter of a method must be definitely assigned before the method returns. Output parameters are typically used in methods that produce multiple return Values.

Example:

```
using System;

namespace CalculatorApplication
{
    class NumberManipulator
    {
        public void getValue(out int x )
        {
            int temp = 5;
            x = temp;
        }

        static void Main(string[] args)
        {
            NumberManipulator n = new NumberManipulator();
            /* local variable definition */
            int a = 100;
```



```

        Console.WriteLine("Before method call, value of a : {0}", a);

        /* calling a function to get the value */
        n.GetValue(out a);

        Console.WriteLine("After method call, value of a : {0}", a);
        Console.ReadLine();
    }
}
}

```

When the above code is compiled and executed, it produces the following result: Before method call, value of a : 100
After method call, value of a : 5

17.5.4 Parameter arrays:

A parameter declared with a params modifier is a parameter array. If a formal parameter list includes a parameter array, it must be the last parameter in the list and it must be of a single-dimensional array type. For example, the types `string[]` and `string[][]` can be used as the type of a parameter array, but the type `string[,]` cannot. It is not possible to combine the params modifier with the ref and out modifiers.

Example:

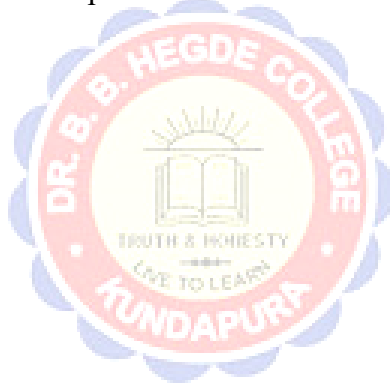
```

using System;

namespace ArrayApplication
{
    class ParamArray
    {
        public int AddElements(params int[] arr)
        {
            int sum = 0;
            foreach (int i in arr)
            {
                sum += i;
            }
            return sum;
        }
    }

    class TestClass
    {
        static void Main(string[] args)
        {
            ParamArray app = new ParamArray();
            int sum = app.AddElements(512, 720, 250, 567, 889);
            Console.WriteLine("The sum is: {0}", sum);
            Console.ReadKey();
        }
    }
}

```



```
    }  
  }  
}
```

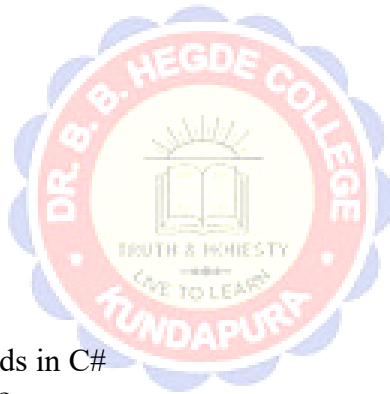
When the above code is compiled and executed, it produces the following result: The sum is: 2938

17.6 Method Overloading:

C# allows us to create more than one method with the same name but with different parameter list and different definition. This is called method overloading. Method overloading is used when methods are required to perform similar tasks but using different input parameters.

Example:

```
public class Class1  
{  
    public void NumbersAdd(int a, int b)  
    {  
        Console.WriteLine(a + b);  
    }  
    public void NumbersAdd(int a, int b, int c)  
    {  
        Console.WriteLine(a + b + c);  
    }  
}
```



Assignment

2 marks

5. Define a method in C#.
6. Give the syntax of declaring methods in C#
7. How do you invoke methods in C#?
8. List the various ways in which parameters can be passed in C# methods.

5 marks

1. Explain the concept of working with methods in C#.
2. Explain pass by value in C#
3. Explain pass by reference in C#